

Examen SGBD



Problema 1

Fie următoare tranzacții:

T1: $read(A);$
 $read(B);$
 if $A = 0$ then $B := B + 1;$
 $write(B).$

T2: $read(B);$
 $read(A);$
 if $B = 0$ then $A := A + 1;$
 $write(A).$

Fie condiția de consistență $A = 0 \wedge B = 0$, cu valorile inițiale $A = B = 0$.

1. Arătați că orice execuție serială a celor două tranzacții păstrează consistența bazei de date.

↳ $T_1 + T_2 = T_1, T_2$ sau T_2, T_1

2. Găsiți cel puțin o execuție concurentă a T1 și T2 ce produce o planificare ne-serIALIZABILĂ.

3. Există o execuție concurentă a T1 și T2 ce produce o planificare serializabilă?

1) Există 2 variante de execuție a T_1 și T_2
 T_1, T_2 sau T_2, T_1

Caz 1: T_1, T_2

	A	B
încăput	0	0
după T_1	0	1
după T_2	0	1

Caz 2: T_2, T_1

	A	B
încăput	0	0
după T_2	1	0
după T_1	1	0

2) O exec. concurentă $T_1, T_2 \Rightarrow$ planificare ne-serIALIZABILĂ

↳ exec. lor nu poate fi o
 non-concurent

$T_1:$
 $read(A)$

$read(B)$

if $A = 0$ then $B := B + 1$

$write(B)$

$T_2:$

$read(B)$
 $read(A)$

if $B = 0$ then $A := A + 1$
 $write(A)$

3) condiția de consistență este $A=0 \vee B=0$

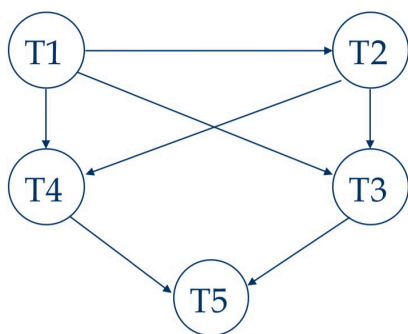
- T_2 trebuie să se execute înainte de finalizarea T_1 , astfel că în orice moment al citirii lui B, înainte de finalizarea lui T_1 , B va avea valoarea 0, astfel ca finalul execuției celor 2 trans. A și B vor avea val. 1.

analog pt T_1 executat înaintea finalizării lui T_2 .

⇒ nu există o exec.

Problema 2

Fie următorul graf de dependență. Este planificarea corespunzătoare acestui graf conflict serializabilă? Justificați răspunsul.



Teorie:

- Planificare serializabilă: același efect asupra BD ca și o planificare serială
- Planificare conflict - serializabilă:
 - același efect asupra BD prin reord. tranzacțiilor

Planificarea corespunzătoare grafului este conflict serializabilă deoarece graful nu are cicluri. O posibilă planificare este: T_1, T_2, T_3, T_4, T_5

Problema 3

Fie următoarele tranzacții :

T1: read(A);

read(B);

if A = 0 then B := B + 1;

write(B).

T2: read(B);

read(A);

if B = 0 then A := A + 1;

write(A).

1. Adăugați instrucțiuni *lock* și *unlock* tranzacțiilor T1 și T2 astfel încât să fie implementat protocolul de blocare în două faze.

2. Este posibil ca execuția tranzacțiilor să intre în *deadlock*?

1) lock - S(A)

read(A)

lock - x(B)

read(B)

⋮

unlock(B)

unlock(A)

lock - S(B)

read(B)

lock - x(A)

read(A)

⋮

unlock(A)

unlock(B)

2)

T1:

lock - S(A)

read(A)

lock - x(B)

⋮

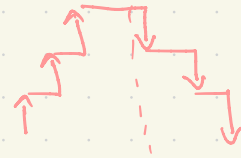
T2:

lock - S(B)

read(B)

lock - x(A)

Q2



→ ord. inv. blocări

lock - S

→ lock shared

= tranz. pot citi,
dar nu pot modifica.

lock - x → lock exclusive

= tranz. citesc și modific.

T1 așteaptă după B

T2 așteaptă după A

T1 ↔ T2

Problema 4

Fie următorul fișier de log:

[start_transaction, T1]	[W, T4, D, 15]
[W, T1, D, 20]	[start_transaction, T3]
[commit, T1]	[W, T3, C, 30]
[checkpoint]	[W, T4, A, 20]
[start_transaction, T2]	[commit, T4]
[W, T2, B, 12]	[W, T2, D, 25]
[start_transaction, T4]	

<- system crash

Descrieți procesul de recuperare a datelor la blocarea sistemului. Specificați ce tranzacții sunt anulate, care operații sunt reexecutate și care sunt anulate.

Pt T_2 și $T_3 \rightarrow$ undo pe tot ce au făcut
Pt $T_4 \rightarrow$ nu e de undo, dacă este cazul, tab.
repetată tranzacția

Anularea operațiilor se face în ordine inversă.

- tranzacția T_1 nu este anulată (și-a dat commit înainte de crash)

- tranzacția T_2 nu și-a dat commit înainte de crash/checkpoint
 $\Rightarrow$ Roll back.

- tranzacția T_3 a început după checkpoint și nu s-a finalizat înainte de crash
 $\Rightarrow$ Rollback

- tranzacția T_4 a început după checkpoint și și-a finalizat exec. înainte de crash $\Rightarrow$ roll forward.

Problema 5

Fie operația join $R \bowtie_{R.a=S.b} S$, executată în următorul context:

- tabela R conține 10.000 înregistrări, cu 10 înreg. pe pagină.
- tabela S conține 2.000 înregistrări, cu 10 înreg. pe pagină.
- atributul b al tabeli S este cheie primară pentru S.
- ambele tabele sunt memorate ca fișiere oarecare.
- nu există indecși definiți pentru nici una dintre tabele.
- sunt disponibile 52 pagini în buffer

Care este cel mai mic cost al joinului lui R și S folosind metodele *page-oriented nested loops*, *block nested loops* și *sort-merge join*? Care este numărul minim de pagini disponibile în buffer pentru a obține același cost?

Page-oriented nested loops join

- îi trebuie minim 3 pagini 2 input 1 output
- minim 3 pag disp în buffer pt a obț. același cost

$$10.000 / 10 = 1000 \text{ înreg. per pagină}$$

$$2.000 / 10 = 200 \text{ înreg. per pagină}$$

$$\text{caz 1: } 1000 + 200 \cdot 1000 = 201000$$

$$\boxed{\text{caz 2: } 200 + 1000 \cdot 200 = 200200} \rightarrow \text{S tabela externă}$$

Block Nested Loops Join

- nu putem cerea mai putine pagini acelasi cost

$$N + M \cdot \left\lceil \frac{N}{B-2} \right\rceil = 200 + 1000 \cdot \underbrace{200/50}_4 = 4200$$

Sort - merge Join

- complexitate alg. merge sort $O(\log_2 n)$

- nr. min. pag Buffer = 3

$$\text{Cost: } 1000 \cdot \log_2 1000 + 200 \cdot \log_2 200 + \underbrace{1200}_{\text{cost readare}}$$

Pt metoda rafinata sort - merge join:

$$3 \cdot (M+N) = 3 \cdot (200+1000) = 3600$$

Nr. min de pag: R - 20 monotonii > 24 pag + 1 de output $\Rightarrow$
S - 4 monotonii $\Rightarrow$ minim de 25 de pag.

Problema 6 - Index

- Creare index pt coloanele folosite frecvent in clause WHERE clauses, conditii JOIN, sau ca parte a Order by.
- Cand e necesar index?

```
Select * from students order by grade;  
Select * from students where grade > 5;
```

create index indexStudents on Students(grade)

Indexul este necesar cand lucram cu un volum mare de date si folosim un numar mare de interogari raportat la restul interogărilor

3. d

4. a: planificare conflict-serializabilă: operațiile conflictuale sunt exec în aceeași ordine
ca într-o planificare serială

↓
nu o operație perie
ambele op. accesează același dat
de date

5. Protocolul WAL → asigură durabilitatea și persistența datelor
R: d

6. c

7. Politica Wait-Die → utilizat în gestionarea concurenței
- cel mai vechi are prioritate

$T_2 > T_1$ T_1 creșcă!

8. c

9. b?

10. a

11.

12. c

13. c

14. c

15. Replicare primară ROWA (Read One Write All)

a

16. b

17. ?

18. d

19. d

20. $A \rightarrow B \leftarrow e \rightarrow B$

b

21. c

22. ?

23. $3(M+N) = 3 \cdot 6000 = 18000$ (d)

24. $100 \cdot \log_2 100 + 1000 \cdot \log_2 1000 + 1100 \sim 3300$ (b)

25. $N + M \cdot (N/(B-2)) = 1100 \cdot 1$

26. $1000 + 1000 \cdot 100 = 101000$?

28. d

29. ?

30. b

31. a

32. A

33. B

34. B

35. c

36. c

37. B

38. A

39. C

40. A

